

Exercise 2: Comprehensive Classification

1. EDA and Feature Engineering

a.

```
# Loading data
setwd("C:/Users/nickz/Desktop/Fall 2025/STAT 747/Homework/Homework 4")
df <- read.csv("financial_transactions.csv")
df$Fraud <- as.factor(df$Fraud)
```

```
tab<-table(df$Fraud)
tab<- rbind(tab,prop.table(tab)) # Table for counts and proportions of labels
rownames(tab)<-c("Count","Proportion")
colnames(tab)<-c("NotFraud_0","Fraud_1")

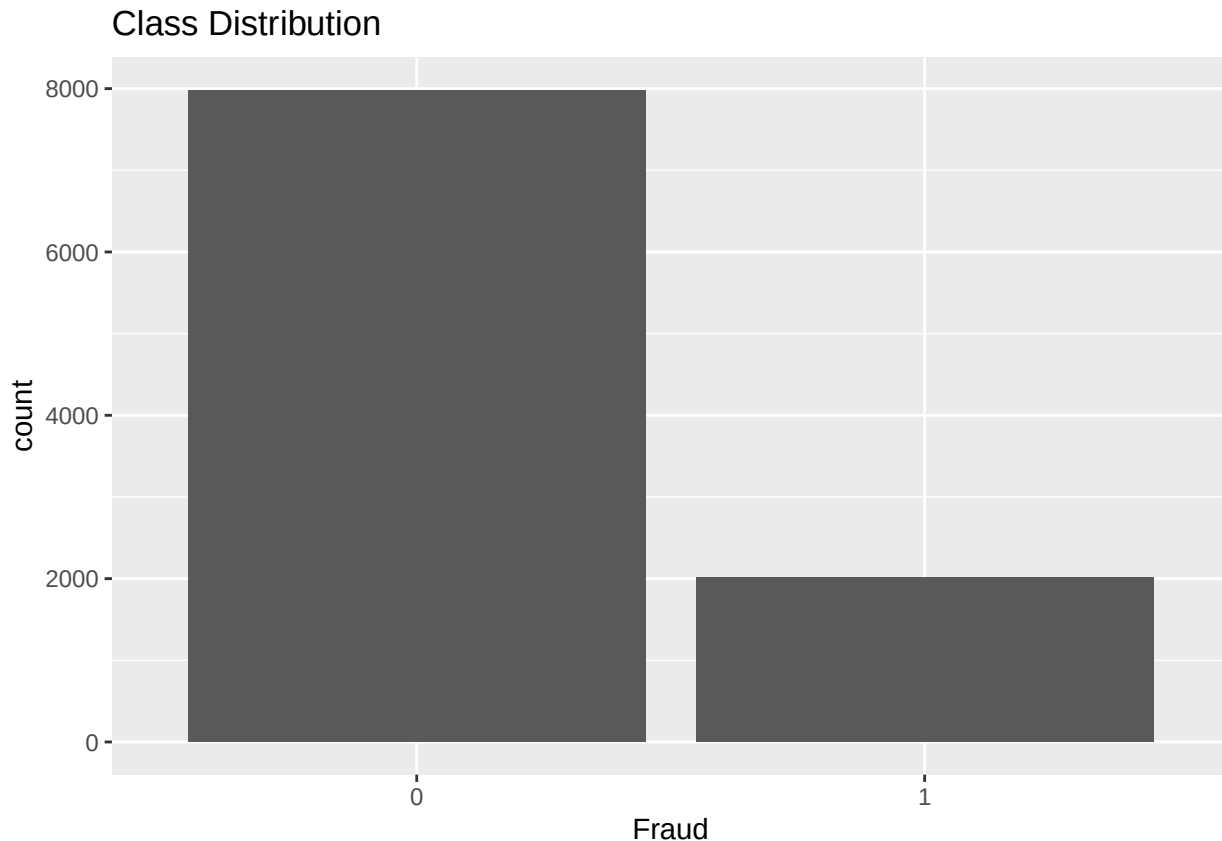
cat("Counts of class labels where 1 signifies fraud and 0 signifies no fraud\n")
```

```
## Counts of class labels where 1 signifies fraud and 0 signifies no fraud
```

```
print(tab)
```

```
##           NotFraud_0   Fraud_1
## Count           7983.0000 2017.0000
## Proportion      0.7983    0.2017
```

```
ggplot(df, aes(x = Fraud)) +
  geom_bar() +
  ggtitle("Class Distribution")
```



We can see that the cases are rather unbalanced with the not-fraud data points having nearly 4 times the number of observations as the fraud data points.

Here are the summary statistics for the independent (all continuous) variables.

```
num_vars <- df %>% select(where(is.numeric)) #only numeric columns (just factors)
summary_stats <- data.frame(
  mean = sapply(num_vars, mean),
  sd    = sapply(num_vars, sd),
  median = sapply(num_vars, median),
  IQR = sapply(num_vars, IQR)
)
print(summary_stats)
```

##	mean	sd	median	IQR
## Amount	0.0128410566	0.9988311	0.0153066129	1.344885
## TimeOfDay	-0.0241436693	1.0093015	-0.0214130326	1.349097
## LocationRisk	0.0075633905	0.9934366	0.0027427071	1.345047
## MerchantScore	0.0050643684	0.9993222	-0.0011904906	1.361032
## UserHistory	-0.0013012003	0.9892729	0.0069956403	1.347846
## DeviceScore	0.0025076630	0.9983112	0.0091040662	1.340749
## TransactionSpeed	-0.0011155103	1.0113619	-0.0026673330	1.360161
## IntlFlag	0.0066102047	1.0033564	0.0179570081	1.346798
## RepeatMerchant	-0.0163997572	0.9885803	-0.0130643250	1.311980
## Velocity_1h	0.0159171375	1.0023406	0.0219807935	1.373697
## Behavior1	-0.0034119251	0.9930245	-0.0067703297	1.357974
## Behavior2	0.0020686440	1.0003225	0.0056129395	1.360563

```

## Behavior3      -0.0003952037  0.9963420 -0.0120703090  1.328916
## Behavior4      0.0089858521  0.9983752  0.0024826359  1.347182
## Behavior5     -0.0038485327  1.0050021 -0.0085121494  1.356047
## Behavior6     -0.0006820124  1.0066725 -0.0044000391  1.383987
## Behavior7      0.0202239153  0.9969095  0.0219156390  1.348356
## Behavior8      0.0124224908  1.0041030  0.0022399442  1.355812
## Behavior9      0.0042168416  1.0063349  0.0074578482  1.354333
## Behavior10     0.0067082977  1.0039877 -0.0009283641  1.346408
## Behavior11     0.0071978168  1.0070737 -0.0089572325  1.352643
## Behavior12    -0.0027650595  1.0080288 -0.0015052833  1.358759
## Behavior13     0.0012420493  1.0017508  0.0042371903  1.345765
## Behavior14     0.0180274133  1.0070706  0.0179403103  1.348988
## Behavior15     0.0110916972  1.0009221  0.0063615404  1.337888
## Behavior16     0.0040663234  1.0066262 -0.0030126714  1.357024
## Behavior17     0.0109803722  0.9985553  0.0033360519  1.345609
## Behavior18    -0.0088343711  1.0171851 -0.0166407657  1.383300
## Behavior19     0.0012588320  0.9944502 -0.0058428187  1.347286
## Behavior20     0.0052027321  0.9985187  0.0076512025  1.367439
## Behavior21     0.0169292080  1.0013780  0.0081372258  1.341488
## Behavior22    -0.0063352777  1.0055972 -0.0032167541  1.345623
## Behavior23    -0.0095153202  1.0136178 -0.0062375665  1.383443
## Behavior24    -0.0145406336  1.0028512 -0.0078239460  1.348931
## Behavior25     0.0018820112  1.0026003  0.0091375546  1.347196
## Behavior26     0.0168780473  1.0173297  0.0161729421  1.334747
## Behavior27    -0.0054477618  0.9927144 -0.0106537972  1.347892
## Behavior28     0.0020548393  0.9994474  0.0054775112  1.347753
## Behavior29    -0.0054299603  0.9952133  0.0245700930  1.366667
## Behavior30     0.0118014938  1.0001126  0.0165608074  1.322935
## Behavior31     0.0052533078  1.0077781 -0.0023941917  1.383785
## Behavior32    -0.0048403719  1.0078308 -0.0086921833  1.356624
## Behavior33     0.0105699126  1.0089978  0.0034661787  1.352692
## Behavior34    -0.0041507167  1.0074602  0.0020044295  1.355138
## Behavior35     0.0099072052  0.9988703  0.0214112561  1.332213
## Behavior36     0.0086378482  0.9948376  0.0125526478  1.340635
## Behavior37     0.0029592291  0.9955726  0.0048962283  1.332170
## Behavior38     0.0093580480  1.0004248  0.0163268838  1.366069
## Behavior39     0.0062026743  0.9944794 -0.0066306949  1.356617
## Behavior40    -0.0015072876  0.9949705  0.0042128650  1.326288

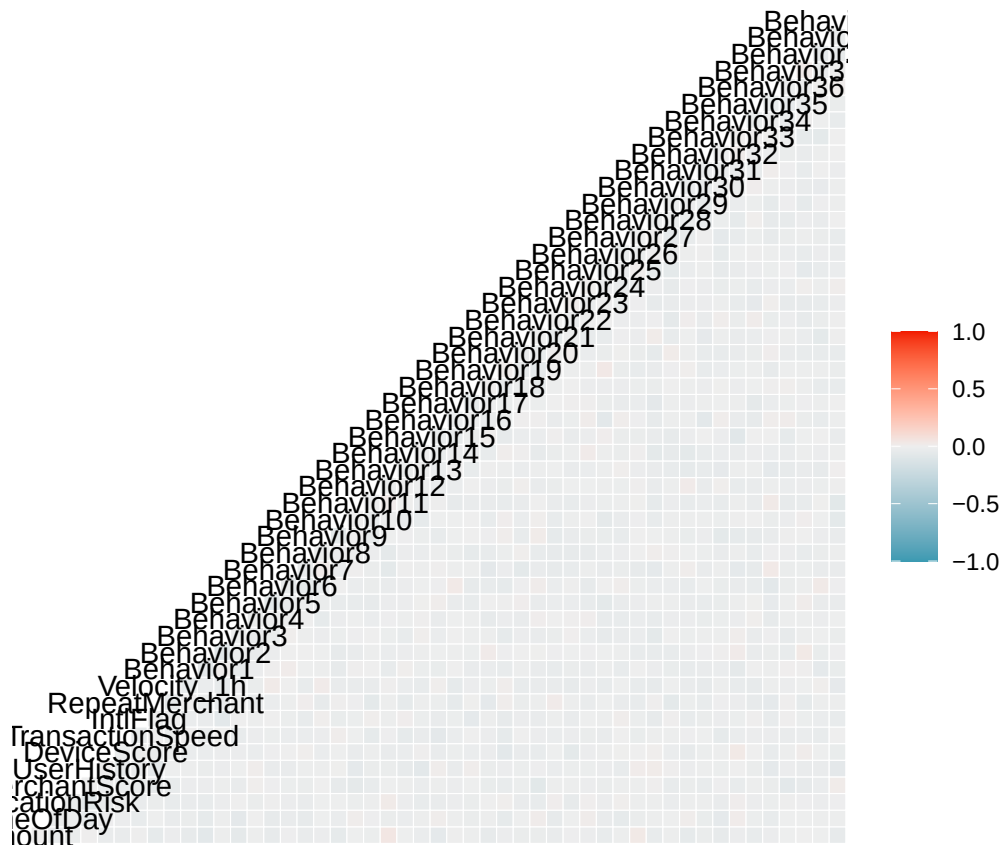
```

```

# Correlation matrix
cor_mat <- cor(num_vars, use = "pairwise.complete.obs")

# Heatmap
ggcorr(cor_mat, label = F)

```



Although this heatmap may be difficult to read because of overlapping labels, it is noticeable that all boxes are a light pink, signifying that there is little-to-no correlation between any of the independent variables.

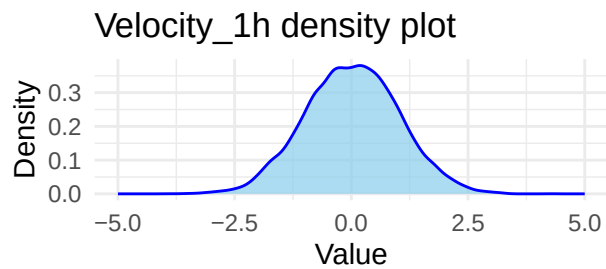
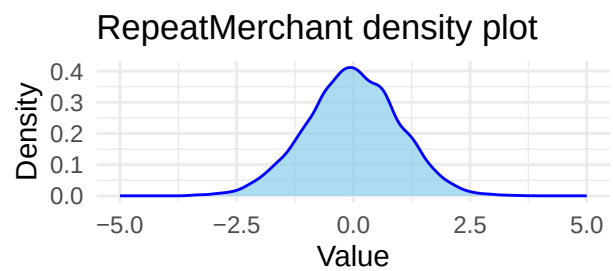
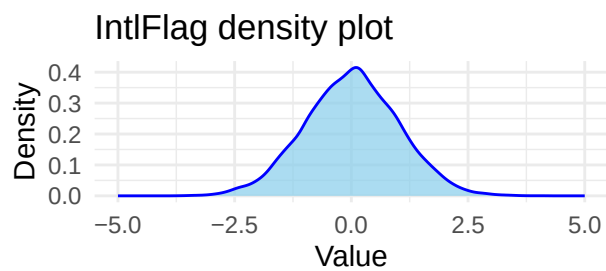
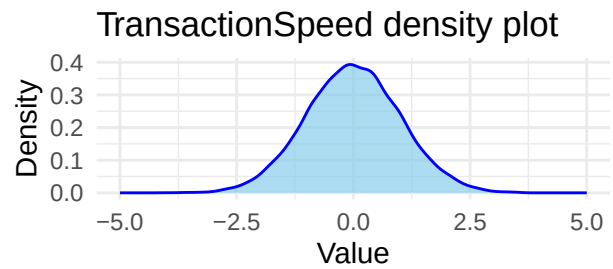
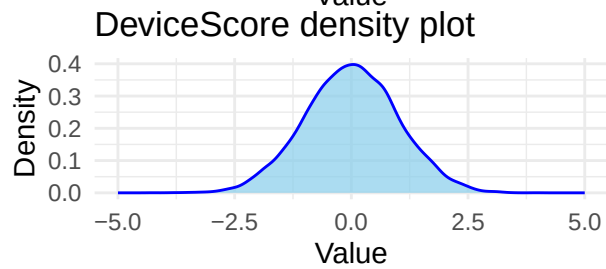
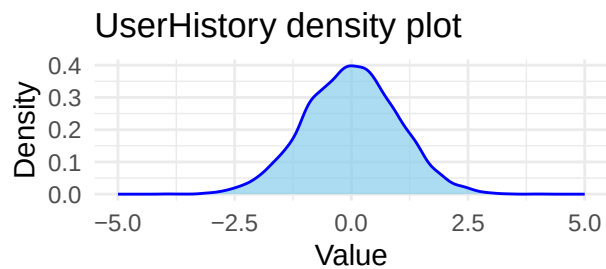
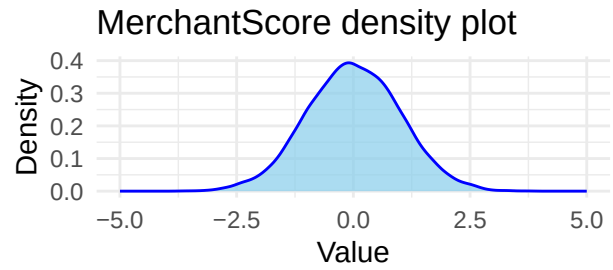
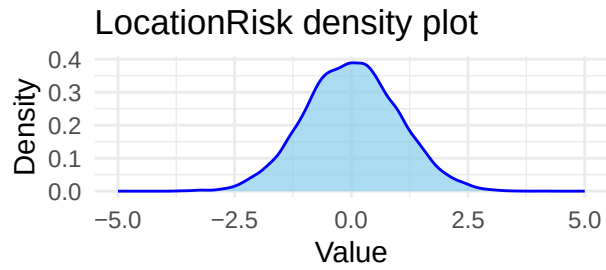
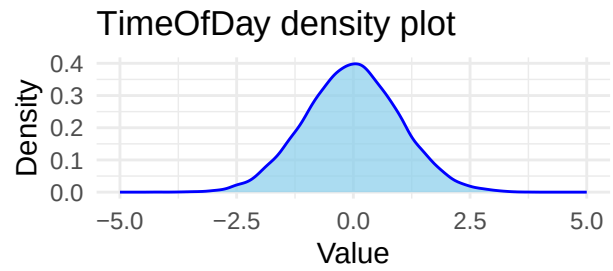
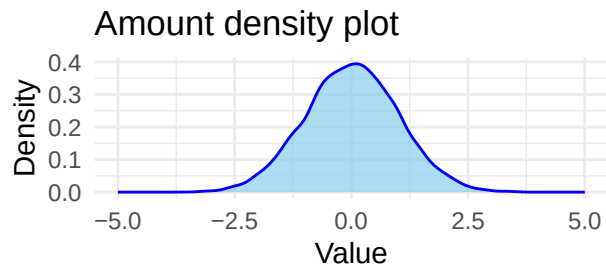
b.

We can conclude from our findings in a. that no interactions terms are necessary.

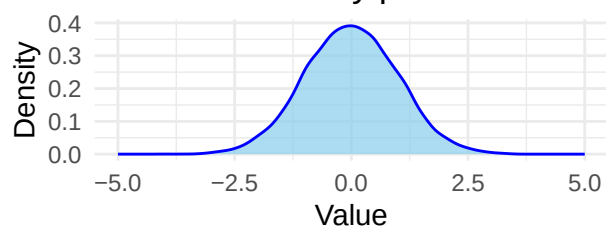
Here are histograms of the factors:

```
num_names <- names(num_vars)
plots <- lapply(num_names, function(v){ # Creating histograms
  ggplot(num_vars, aes(x=num_vars[,v])) +
    geom_density(fill = "skyblue", color = "blue", alpha = 0.7) +
    labs(title=paste(v, "density plot"),
         x = "Value",
         y = "Density") +
    xlim(-5,5) +
    theme_minimal()
})

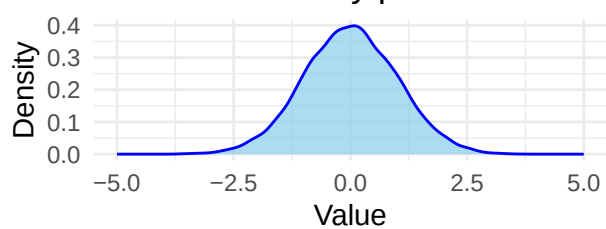
for (i in 1:10){ # Printing histograms
  grid.arrange(grobs=plots[(5*i-4):(i*5)], ncol=2)
}
```



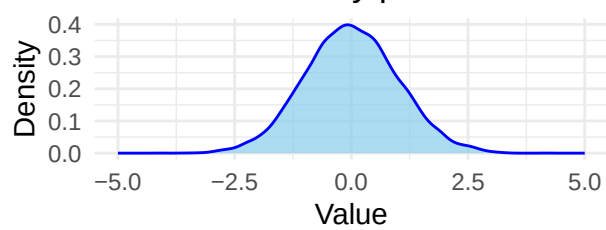
Behavior1 density plot



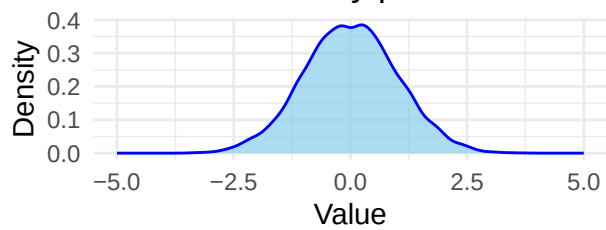
Behavior2 density plot



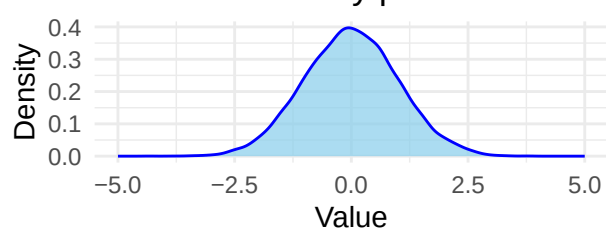
Behavior3 density plot



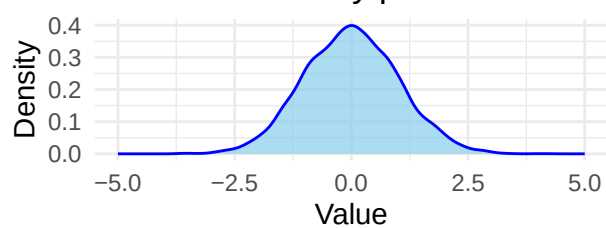
Behavior4 density plot



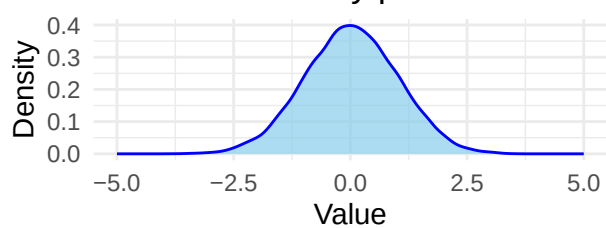
Behavior5 density plot



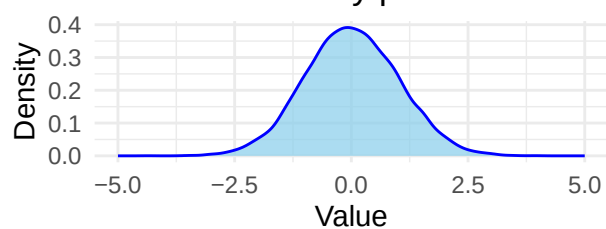
Behavior6 density plot



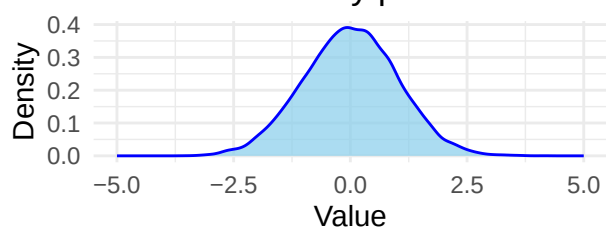
Behavior7 density plot



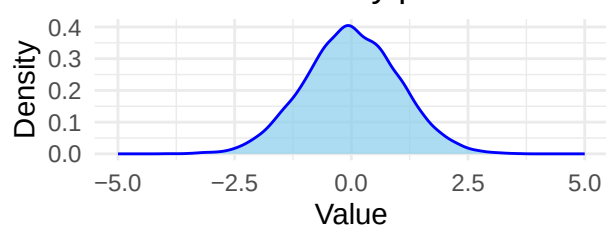
Behavior8 density plot



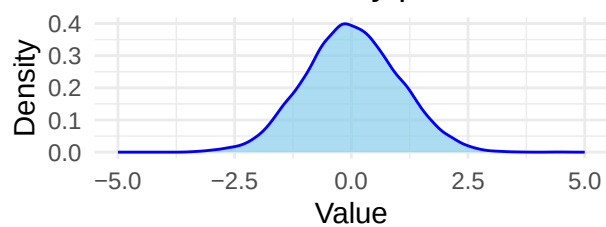
Behavior9 density plot



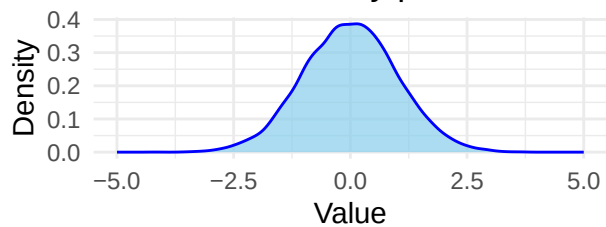
Behavior10 density plot



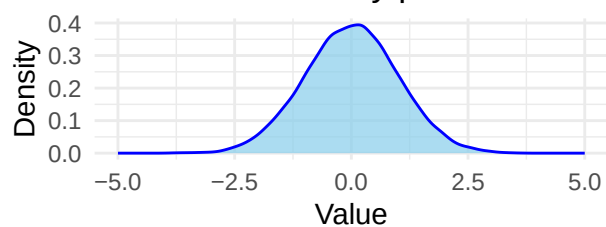
Behavior11 density plot



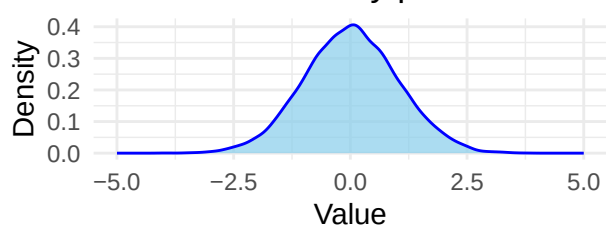
Behavior12 density plot



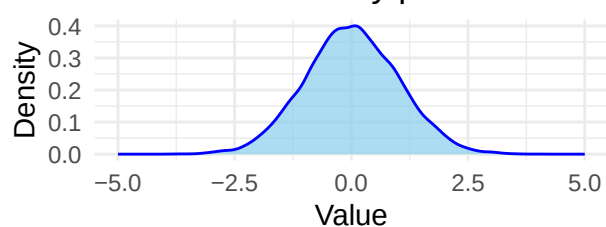
Behavior13 density plot



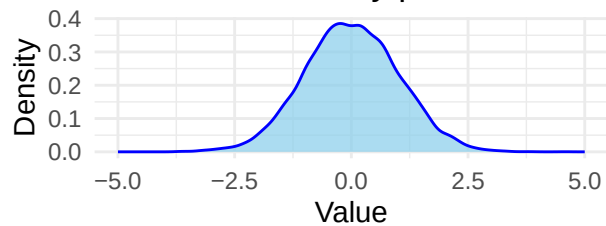
Behavior14 density plot



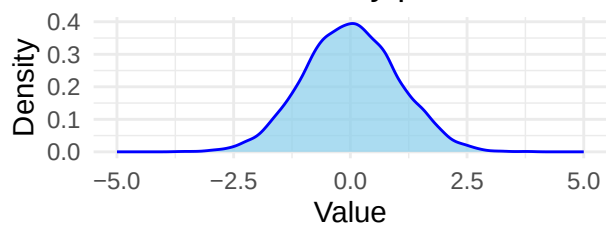
Behavior15 density plot



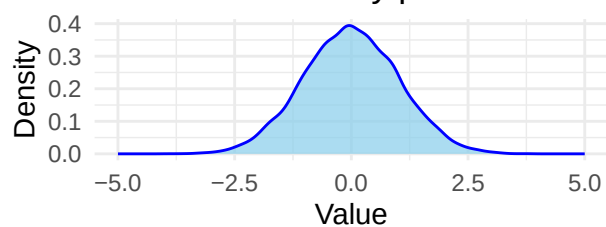
Behavior16 density plot



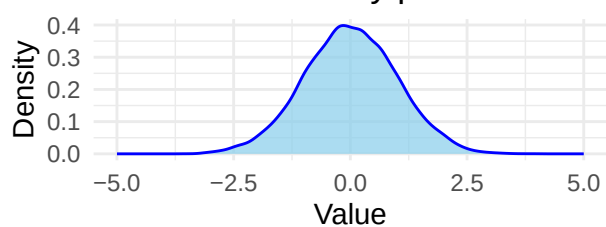
Behavior17 density plot



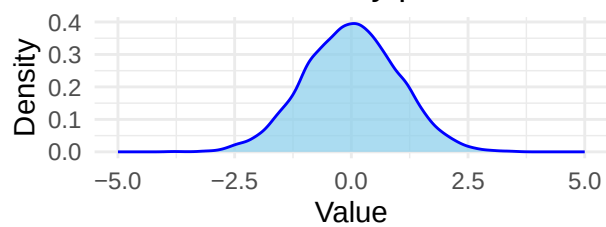
Behavior18 density plot



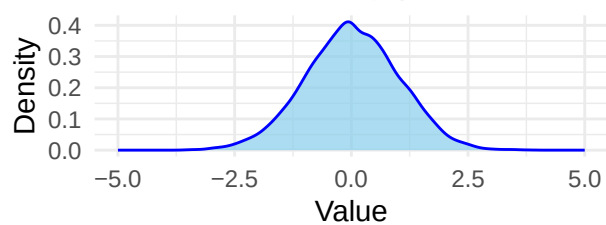
Behavior19 density plot



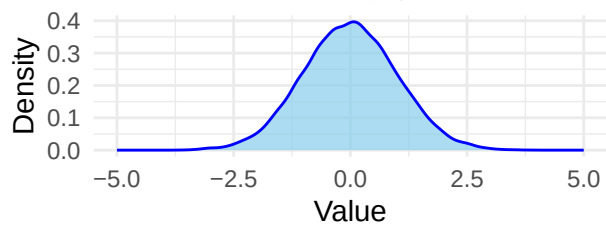
Behavior20 density plot



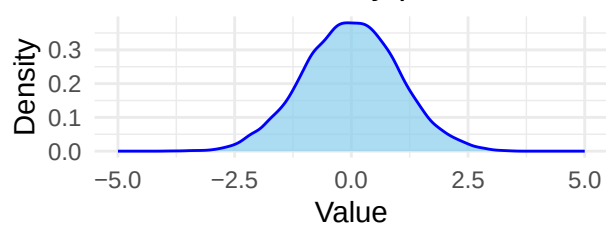
Behavior21 density plot



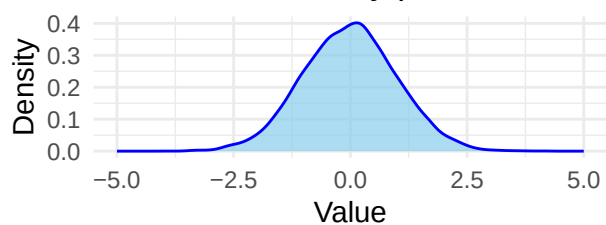
Behavior22 density plot



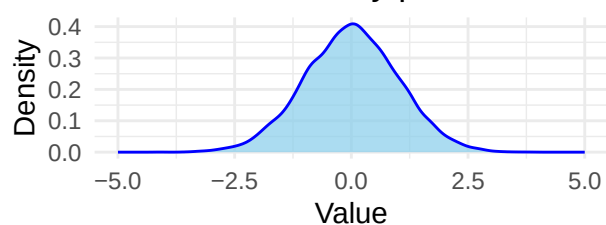
Behavior23 density plot



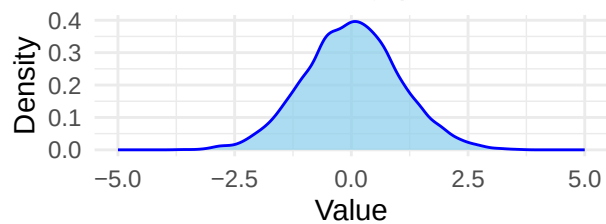
Behavior24 density plot



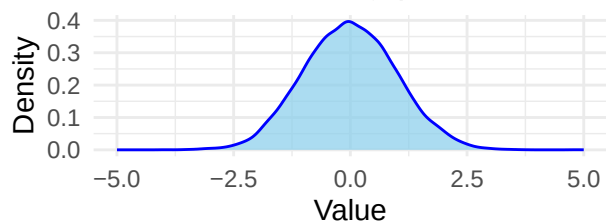
Behavior25 density plot



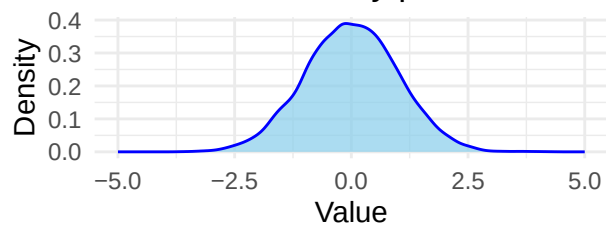
Behavior26 density plot



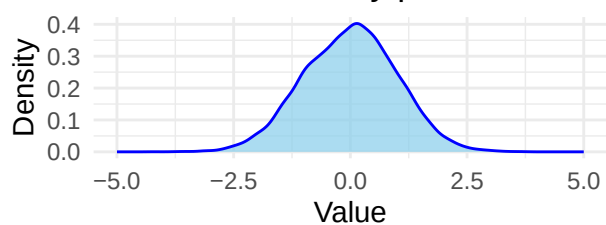
Behavior27 density plot



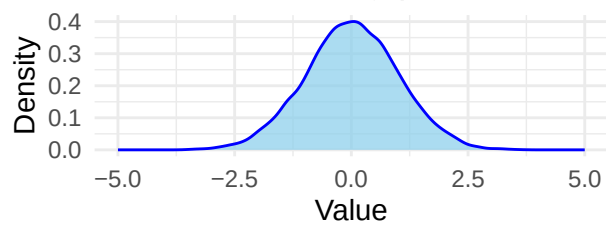
Behavior28 density plot

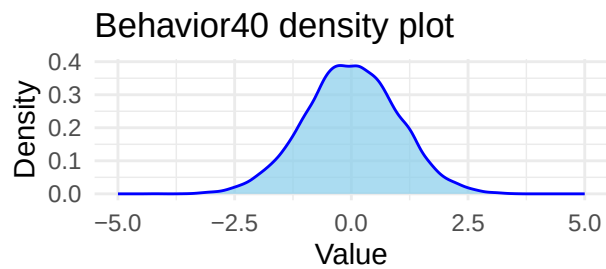
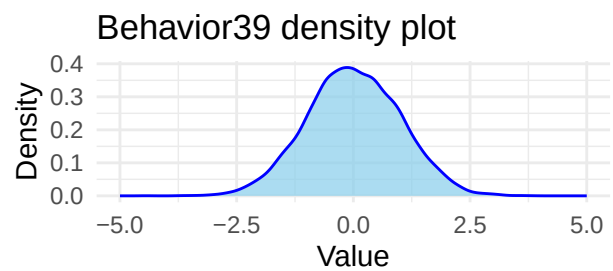
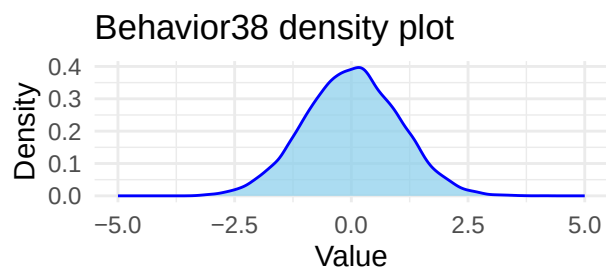
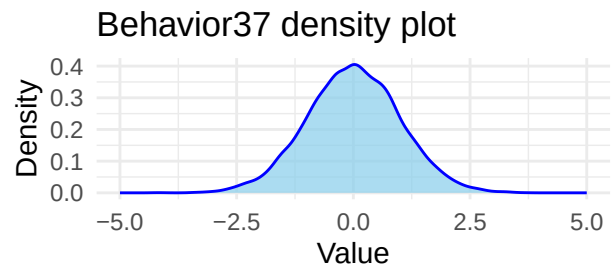
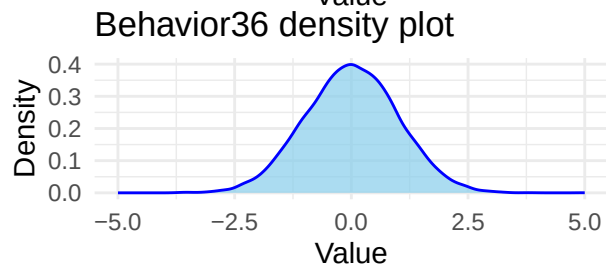
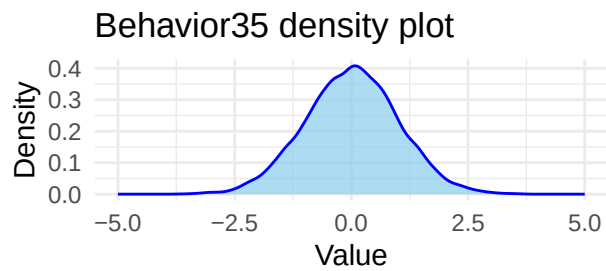
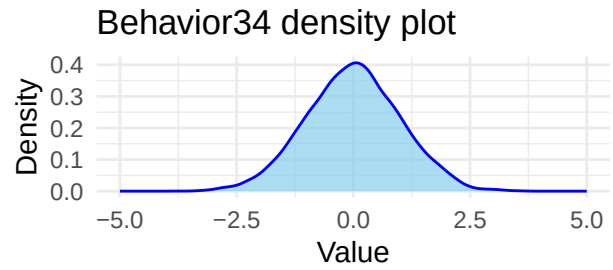
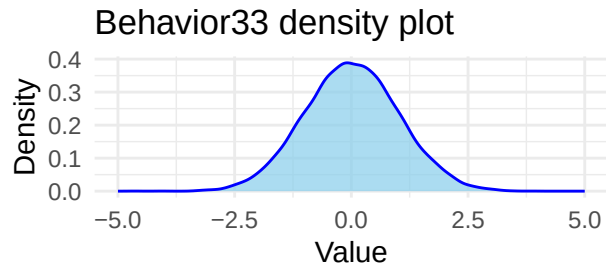
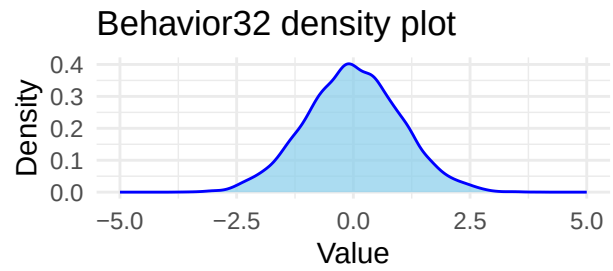
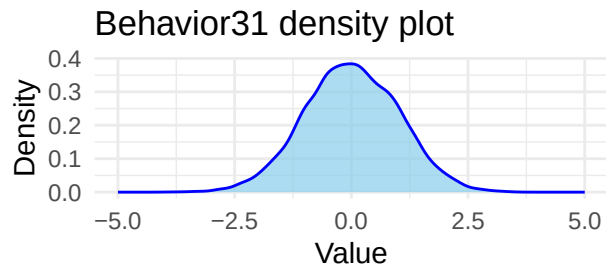


Behavior29 density plot



Behavior30 density plot





We can see that all factors are relatively normally distributed and centered around 5. There is really not

much need for transformations, but as a means of preprocessing, we centered the data about their means and scale the data by their standard deviations for each independent variable.

```
df.raw <- df # Saving raw data

# Transforming data by centering and scaling the independent variable columns
df <- cbind.data.frame(Fraud=df$Fraud,
                       scale_data_frame(num_vars, center = T, scale = T))
```

c.

```
# Applying SMOTE using 5 NN to synthetically generate additional data points for
# fraud class 1.
df.smote <- SMOTE(df[, -1], df[, 1])
df_smoted <- df.smote$data
df_smoted$class <- as.factor(df_smoted$class)
```

2. Model Specification & Training

a.

```
# Easier format of data for some machines' training functions
x <- model.matrix(class ~ ., df_smoted)[, -1]
y <- df_smoted$class

# Logistic Regression
cv_logit <- cv.glmnet(x, y, family = "binomial", alpha = 1)

# Random Forest
rf_model <- randomForest(class ~ ., data = df_smoted,
                          ntree = 500, mtry = floor(sqrt(ncol(df_smoted)-1)))

# Gradient Boosting Machine
dtrain <- xgb.DMatrix(data = x, label = as.numeric(y)-1) # formatting for GBM
params <- list( # parameter list for GBM
  objective = "binary:logistic",
  eval_metric = "auc",
  eta = 0.1, max_depth = 5
)
xgb_model <- xgb.train(params, dtrain, nrounds = 300) # Fitting model

# SVM
svm_model <- svm(class ~ ., data = df_smoted, kernel = "radial", cost = 1,
                 gamma = 0.01)
```

b.

```
mlp_model <- caret::train(x=x,
  y=y,
  method="mlpML")
```

c.

Logistic Regression: Logistic regression can be formulated as:

For a binary outcome $y_i \in \{0, 1\}$:

$$p_i = \Pr(y_i = 1 \mid x_i) = \frac{1}{1 + \exp(-x_i^\top \beta)}$$

We have the negative log likelihood

$$\ell(\beta) = - \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

$$\min_{\beta} \ell(\beta) + \lambda \left[(1 - \alpha) \frac{1}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1 \right]$$

Where the tuning parameter $\lambda \geq 0$ controls regularization strength, and $\alpha \in [0, 1]$ controls the type of regularization ($\alpha = 0$: Ridge (L2), $\alpha = 1$: Lasso (L1). $0 < \alpha < 1$: Elastic Net (L1+L2)).

Random forest: Each tree is trained on a bootstrap sample and each split only considers a random subset of predictors. If we call

$$\hat{y}^{(b)}(x) = \text{prediction from tree } b,$$

Then RF is the majority vote

$$\hat{y}(x) = \text{mode} \left\{ \hat{y}^{(1)}(x), \hat{y}^{(2)}(x), \dots, \hat{y}^{(B)}(x) \right\}.$$

For m features randomly selected at each split from the full set of p features,

$$m \leq p, \quad \mathcal{M}_{\text{split}} \subseteq \{1, \dots, p\}, \quad |\mathcal{M}_{\text{split}}| = m$$

The split is chosen by impurity via Gini index

$$\theta^* = \arg \min_{\theta \in \mathcal{M}_{\text{split}}} \text{Gini}(\theta).$$

Such that

$$\text{Gini}(t) = \sum_{k=1}^K p_k(t) (1 - p_k(t)).$$

Where $p_k(t)$ is the proportion of class k in node t .

The tuning parameters for RF classification are the number of trees (B or M), the number of variables at each split (m), the leaf size (minimum node size $n_{\min}$), maximum tree depth ($d_{\max}$), and sample size per tree for each bootstrap sample (b_{boot}).

Gradient Boosting:

Gradient boosting builds a classifier by sequentially adding weak learner decision trees such that,

$$F_M(x) = \sum_{m=1}^M \nu f_m(x)$$

where, $f_m(x)$ is a decision tree at iteration m , $\nu \in (0, 1]$ is the learning rate, and M is the number of iterations/trees.

In the binary case, the predicted probability for class 1 is

$$p(x) = \frac{1}{1 + e^{-F_M(x)}}.$$

The GBM minimizes logistic loss, which can be written as

$$L = \sum_{i=1}^n \log \left(1 + e^{-y_i F(x_i)} \right)$$

where y_i is coded such that $y_i \in \{-1, +1\}$

Each iteration fits a tree to the negative gradient using pseudo-residuals such that

$$r_{im} = -\frac{\partial L}{\partial F_{m-1}(x_i)},$$

Which, for logistic loss is:

$$r_{im} = y_i - p_{m-1}(x_i).$$

Then a regression tree $f_m(x)$ is fit to these residuals and the model is updated such that

$$F_m(x) = F_{m-1}(x) + \nu f_m(x)$$

The tuning parameters are the number of boosting iterations/weak learner trees M , the learning rate $0 < \nu < 1$, the tree depth $d_{\max}$, the minimum observations per leaf, the fraction of bag sampled, and the type and strength of regularization which is controlled by λ and α like in logistic regression.

SVM: From homework 2, the dual formulation of SVM for binary classification is

$$\max_{\alpha} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^{\top} x_j$$

$$\text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad i \in [n]$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

The Lagrange multipliers α_i are associated with margin constraints in that they measure the influence of training point i on the optimal hyperplane. The Karush-Kuhn-Tucker conditions we derived imply three non-linearly separable cases based on these conditions:

$$\alpha_i = 0 \Rightarrow y_i(w^\top x_i + b) \geq 1 \quad \text{and} \quad \xi_i = 0 \quad (1)$$

$$0 < \alpha_i < C \Rightarrow y_i(w^\top x_i + b) = 1 \quad \text{and} \quad \xi_i = 0 \quad (2)$$

$$\alpha_i = C \Rightarrow y_i(w^\top x_i + b) \leq 1 \quad \text{and} \quad \xi_i \geq 0 \quad (3)$$

Only points with $\alpha_i > 0$ contribute to the weight vector $w = \sum_{i=1}^n \alpha_i y_i x_i$; these are the support vectors.

MLP: For neural networks like MLP, the parameters are The q p -dimensional projection vectors $\beta_1, \dots, \beta_q$. The q scalars $\beta_{01}, \dots, \beta_{0q}$ known as shifts. The q coefficient scalars for terms $\gamma_1, \dots, \gamma_q$. The intercept γ_0 .

Where the the weights w are dedigned as $w = \{\beta_j, \gamma_j, \beta_{0j}, \gamma_0\}$, and the output is referred to as $\text{net}(x, w) = \mu(x) = \phi(\eta(x))$.

Sequentially, each node j of the hidden layer computes the linear combination h_j given a p -dimensional input vector $x = (x_1, x_2, \dots, x_p)^\top$,

$$h_j(x) = \beta_{0j} + \beta_{1j}x_1 + \beta_{2j}x_2 + \dots + \beta_{pj}x_p$$

Then, that linear combination is passed through an activation function $\psi(\cdot)$, which in the case of MLP is usually a sigmoidal function $\sigma(\cdot)$.

$$z_j = \psi(h_j(x)) = \psi\left(\beta_{0j} + \sum_{\ell=1}^p \beta_{\ell j}x_\ell\right)$$

These z_j values are combined linearly with q real coefficients $\gamma_1, \dots, \gamma_q$ and the intercept term γ_0 is added such that

$$\eta(x) = \gamma_0 + \sum_{j=1}^q \gamma_j \psi\left(\beta_{0j} + \sum_{\ell=1}^p \beta_{\ell j}x_\ell\right)$$

Then, the value $\eta(x)$ is passed to the output layer using the output activation function (also usually a sigmoidal function, but this depends on the particular classification; in the case of this data using 0 and 1 as labels, the heaviside step function could be used, but factor coding is arbitrary as long as the output is interpretable).

$$\mu(x) = \phi(\eta(x)) = \phi\left(\gamma_0 + \sum_{j=1}^q \gamma_j \psi\left(\beta_{0j} + x^\top \beta_j\right)\right)$$

Tuning parameters are number of hidden layers L , neurons per layer h_ℓ , the activation function ψ . Additionally, for loss-oriented training, sometimes a learning rate (ν or η) is specified, as well as regularization penalty λ , a parameter for batch size (number of samples to work through before updating internal model parameters (one forward and one backward pass of the neural network)), and number of epochs (number of times the entire dataset has to be worked through the learning algorithm).

3. Model Selection & Tuning

a.

*# Training and tuning were taking too long to be viable, so here I am reducing
the training dataset to a class-balanced 2000 observations and the testing
set to a class-balanced 250 observations.*

```
set.seed(747)
```

```
train.test_split <- initial_split(df_smoted)
train <- training(train.test_split)
test <- testing(train.test_split)
```

```
train. <- train %>% group_by(class) %>% sample_n(1000)
test. <- test %>% group_by(class) %>% sample_n(250)
```

Easier format of data for some machines' training functions

```
x <- model.matrix(class ~ ., train.)[, -1]
y <- train.$class
```

```
logit_cv <- nestcv.glmnet(y=y,
  x=x,
  family = "binomial",
  outer_method = "cv",
  n_outer_folds = 5,
  n_inner_folds = 10,
  alphaSet = seq(0.1,1,0.1),
  outer_train_predict = TRUE,
  cv.cores = 5,
  keep = TRUE,
  finalCV = TRUE)
```

```
y_tr <- factor(ifelse(as.numeric(y)-1==1,"Fraud","NotFraud"))
```

```
rf_cv <- nestcv.train(x=x,
  y=y_tr,
  method="rf",
  outer_method = "cv",
  n_outer_folds = 5,
  n_inner_folds = 10,
  outer_train_predict = TRUE,
  savePredictions = "final",
  finalCV = TRUE
)
```

```
gbm_cv <- nestcv.train(x=x,
  y=y_tr,
  method="xgbTree",
  outer_method = "cv",
```

```

        n_outer_folds = 5,
        n_inner_folds = 10,
        outer_train_predict = TRUE,
        savePredictions = "final",
        keep = TRUE,
        finalCV = TRUE
    )

svm_cv <- nestcv.train(x=as.matrix(x),
    y=y_tr,
    method="svmRadial",
    outer_method = "cv",
    n_outer_folds = 3,
    outer_train_predict = TRUE,
    finalCV = TRUE,
    savePredictions = "final",
    keep = TRUE,
    type = "C-svc",
    trControl = trainControl(number = 5,
                             method = "cv",
                             classProbs = TRUE,
                             summaryFunction = twoClassSummary,
                             savePredictions = "final")
    )

mlp_cv <- nestcv.train(x=x,
    y=y_tr,
    method="mlpML",
    outer_method = "cv",
    n_outer_folds = 5,
    n_inner_folds = 10,
    outer_train_predict = TRUE,
    savePredictions = "final",
    keep = TRUE,
    finalCV = TRUE
    )

```

```

# Producing predictions on the training set for the best models and using them
# to calculate specificity and sensitivity for part c
logit_cv_preds<- predict(logit_cv$final_fit,
    newx = as.matrix(test.[,-51]),type="response") %>%
    {ifelse(. >= 0.5,1,0)} |> as.factor()
logit_cv_preds <- cbind.data.frame(truth=test.$class,estimate=logit_cv_preds)

rf_cv_preds<- predict(rf_cv$final_fit,
    newdata = test.[,-51],type="raw") %>%
    {ifelse(. == "Fraud",1,0)} |> as.factor()
rf_cv_preds <- cbind.data.frame(truth=test.$class,estimate=rf_cv_preds)

gbm_cv_preds<- predict(gbm_cv$final_fit,
    newdata = test.[,-51],type="raw") %>%
    {ifelse(. == "Fraud",1,0)} |> as.factor()
gbm_cv_preds <- cbind.data.frame(truth=test.$class,estimate=gbm_cv_preds)

```

```

svm_cv_preds<- predict(svm_cv$final_fit,
  newdata = test.[,-51],type="raw") %>%
  {ifelse(. == "Fraud",1,0)} |> as.factor()
svm_cv_preds <- cbind.data.frame(truth=test.$class,estimate=svm_cv_preds)

mlp_cv_preds<- predict(mlp_cv$final_fit,
  newdata = test.[,-51],type="raw") %>%
  {ifelse(. == "Fraud",1,0)} |> as.factor()
mlp_cv_preds <- cbind.data.frame(truth=test.$class,estimate=mlp_cv_preds)

# Data frame of sensitivities and specificities
multi_metric_cv <- rbind.data.frame(
  c(sens(logit_cv_preds, truth=truth, estimate = estimate)[,3],
    spec(logit_cv_preds, truth=truth, estimate = estimate)[,3]),
  c(sens(svm_cv_preds, truth=truth, estimate = estimate)[,3],
    spec(svm_cv_preds, truth=truth, estimate = estimate)[,3]),
  c(sens(rf_cv_preds, truth=truth, estimate = estimate)[,3],
    spec(rf_cv_preds, truth=truth, estimate = estimate)[,3]),
  c(sens(gbm_cv_preds, truth=truth, estimate = estimate)[,3],
    spec(gbm_cv_preds, truth=truth, estimate = estimate)[,3]),
  c(sens(mlp_cv_preds, truth=truth, estimate = estimate)[,3],
    spec(mlp_cv_preds, truth=truth, estimate = estimate)[,3]))

row.names(multi_metric_cv) <- c("Logit","SVM.RBF","RF","GBM","MLP")
colnames(multi_metric_cv) <- c("Sensitivity","Specificity")

```

b

```

# CV folds for Bayes setup
folds <- vfold_cv(train., v = 5, strata = class)

# Recipe setup
rec <- recipe(class ~ ., data = train.) |>
  step_normalize(all_numeric_predictors())

logreg_spec <- logistic_reg(
  penalty = tune(), # regularization strength
  mixture = tune() # 0 = ridge, 1 = lasso
) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

svm_spec <- svm_rbf(
  cost = tune(),
  rbf_sigma = tune(),
  margin = tune()
) |>
  set_engine("kernlab") |>
  set_mode("classification")

rf_spec <- rand_forest(

```



```

mtry = tune(),
trees = tune(),
min_n = tune()
) |>
  set_engine("ranger") |>
  set_mode("classification")

gbm_spec <- boost_tree(
  trees = tune(),
  learn_rate = tune(),
  tree_depth = tune(),
  loss_reduction = tune(),
  min_n = tune(),
  sample_size = tune()
) |>
  set_engine("xgboost") |>
  set_mode("classification")

mlp_spec <- parsnip::mlp(
  hidden_units = tune(),
  penalty = tune(),
  epochs = tune()
) |>
  set_engine("nnet") |>
  set_mode("classification")

```

```

# workflows
logreg_wf <- workflow() |> add_model(logreg_spec) |> add_recipe(rec)
svm_wf <- workflow() |> add_model(svm_spec) |> add_recipe(rec)
rf_wf <- workflow() |> add_model(rf_spec) |> add_recipe(rec)
gbm_wf <- workflow() |> add_model(gbm_spec) |> add_recipe(rec)
mlp_wf <- workflow() |> add_model(mlp_spec) |> add_recipe(rec)

```

```

# control parameters for bayes tuning
ctrl <- control_bayes(
  verbose = TRUE,
  no_improve = 10, # early stop
  save_workflow = TRUE
)

```

```

set.seed(747)

# Bayes parameter optimization for all models
logreg_bayes <- tune_bayes(
  logreg_wf,
  resamples = folds,
  initial = 5,
  iter = 15,
  control = ctrl,
  metrics = metric_set(roc_auc, sens, yardstick::spec)
)

# Best model for logit

```

```

logreg_best <- select_best(logreg_bayes, metric = "sens")
# Final workflow (logit)
logreg_final_wf <- finalize_workflow(logreg_wf, logreg_best) |> fit(train)

# predictions (logit)
logreg_preds <- logreg_final_wf |>
  predict(test, type = "prob") |>
  bind_cols(predict(logreg_final_wf, test)) |>
  bind_cols(test)

##### Process is repeated for other models

svm_bayes <- svm_wf |>
  tune_bayes(
    resamples = folds,
    initial = 5,
    iter = 5,
    control = ctrl,
    metrics = metric_set(roc_auc, sens, yardstick::spec)
  )

svm_best <- select_best(svm_bayes, metric = "sens")
svm_final_wf <- finalize_workflow(svm_wf, svm_best) |> fit(train.)
svm_final_fit <- fit(svm_final_wf, train.)

svm_preds <- svm_final_fit |>
  predict(test., type = "prob") |>
  bind_cols(predict(svm_final_fit, test.)) |>
  bind_cols(test.)

rf_params <- extract_parameter_set_dials(rf_wf) |>
  finalize(select(train, -class))

rf_bayes <- rf_wf |>
  tune_bayes(
    resamples = folds,
    initial = 5,
    iter = 5,
    param_info = rf_params,
    control = ctrl,
    metrics = metric_set(roc_auc, sens, yardstick::spec)
  )

rf_best <- select_best(rf_bayes, metric = "sens")
rf_final_wf <- finalize_workflow(rf_wf, rf_best) |> fit(train)

rf_final_fit <- fit(rf_final_wf, train.)

rf_preds <- rf_final_fit |>
  predict(test., type = "prob") |>
  bind_cols(predict(rf_final_fit, test.)) |>
  bind_cols(test.)

```

```

gbm_bayes <- gbm_wf |>
  tune_bayes(
    resamples = folds,
    initial = 10,
    iter = 10,
    control = ctrl,
    metrics = metric_set(roc_auc, sens, yardstick::spec)
  )
gbm_best <- select_best(gbm_bayes, metric = "sens")

gmb_final_wf <- finalize_workflow(gbm_wf, gbm_best) |> fit(train)

gbm_final_fit <- fit(gmb_final_wf, train.)

gbm_preds <- gbm_final_fit |>
  predict(test., type = "prob") |>
  bind_cols(predict(gbm_final_fit, test.)) |>
  bind_cols(test.)

mlp_bayes <- mlp_wf |>
  tune_bayes(
    resamples = folds,
    initial = 5,
    iter = 5,
    control = ctrl,
    metrics = metric_set(roc_auc, sens, yardstick::spec)
  )
mlp_best <- select_best(mlp_bayes, metric = "sens")
mlp_final_wf <- finalize_workflow(mlp_wf, mlp_best) |> fit(train)

mlp_final_fit <- fit(mlp_final_wf, train.)

mlp_preds <- mlp_final_fit |>
  predict(test., type = "prob") |>
  bind_cols(predict(mlp_final_fit, test.)) |>
  bind_cols(test.)

```

```

# Computing sensitivity and specificity for all models tuned by bayes optimization
multi_metric<-rbind.data.frame(c(sens(logreg_preds, truth = class, .pred_class)[,3],
  yardstick::spec(logreg_preds, truth = class, .pred_class)[,3]
),
  c(sens(svm_preds, truth = class, .pred_class)[,3],
    yardstick::spec(svm_preds, truth = class, .pred_class)[,3]),
  c(sens(rf_preds, truth = class, .pred_class)[,3],
    yardstick::spec(rf_preds, truth = class, .pred_class)[,3]),
  c(sens(gbm_preds, truth = class, .pred_class)[,3],
    yardstick::spec(gbm_preds, truth = class, .pred_class)[,3]),
  c(sens(mlp_preds, truth = class, .pred_class)[,3],
    yardstick::spec(mlp_preds, truth = class, .pred_class)[,3]))
# These are sensitivity and specificity for bayes models

row.names(multi_metric) <- c("Logit", "SVM.RBF", "RF", "GBM", "MLP")
colnames(multi_metric) <- c("Sensitivity", "Specificity")

```

```
# Comparing CV vs Bayes for Sensitivity and Specificity
cat("Sensitivity and Specificity for Nested-CV-Tuned Models \n")
```

```
## Sensitivity and Specificity for Nested-CV-Tuned Models
```

```
print(multi_metric_cv)
```

```
##           Sensitivity Specificity
## Logit           0.784         0.868
## SVM.RBF         0.816         0.864
## RF              0.824         0.840
## GBM             0.808         0.872
## MLP             0.784         0.880
```

```
cat("Sensitivity and Specificity for Bayes-Optimization-Tuned Models \n")
```

```
## Sensitivity and Specificity for Bayes-Optimization-Tuned Models
```

```
print(round(multi_metric,3))
```

```
##           Sensitivity Specificity
## Logit           0.845         0.818
## SVM.RBF         1.000         0.156
## RF              0.880         0.804
## GBM             0.816         0.868
## MLP             0.800         0.860
```

Overall, the Bayes-optimization-tuned models performed better. I was particularly interested in model sensitivity in this situation because correctly detecting fraud is more important than correctly identifying lack of fraud.

4. Uncertainty & Interpretation

a.

The best performing model before was SVM tuned with Bayes optimization, so I used that for this section. The analogue for conformal prediction intervals for classification is conformal test sets. These are a set of labels the conformal predictor cannot reject with (here) 95% confidence. This is formed for SVM RBF below:

```
library(probably)
set.seed(746)
# Conformance data
cal_data <- train %>% group_by(class) %>% sample_n(250)

# The predictor data for calibration
cal_predictors <- cal_data[,-51]
# The true outcome for calibration
cal_outcome <- cal_data$class
```

```

# Model predictions on conformal data
cal_probs_svm<- predict(svm_final_fit,
  new_data = cal_predictors,type="prob")$.pred_1

quantile_level <- ceiling(501 * 0.95) / 500 # q hat quantile level

q_hat <- quantile(cal_probs_svm, probs = quantile_level) # q hat

min_prob_threshold <- 1-q_hat

test_probs_svm <- predict(svm_final_fit, new_data = test.[,-51], type = "prob")

prediction_sets_svm <- test_probs_svm %>%
  # 1. Add a unique ID for grouping rows later
  # Use rowid_to_column() for a robust ID creation
  tibble::rowid_to_column("obs_id") %>%

  # 2. Convert to a long format: one row per observation-class pair
  pivot_longer(
    # Use a selection helper to pick only the probability columns (the numerics)
    cols = where(is.numeric) & !obs_id,
    names_to = "Class_Name",
    values_to = "Probability"
  ) %>%

  # 3. Identify classes that belong in the prediction set
  mutate(
    # TRUE if P(k) > min_prob_threshold
    is_in_set = Probability > min_prob_threshold
  ) %>%

  # 4. Group by the original observation ID
  group_by(obs_id) %>%

  # 5. Aggregate: create a list of class names where is_in_set is TRUE
  summarise(
    pred_set = list(Class_Name[is_in_set]),
    .groups = "drop" # Drop the grouping at the end
  ) %>%

  # 6. Pull the resulting list column
  pull(pred_set)

print(head(prediction_sets_svm))

```

```

## [[1]]
## [1] ".pred_0" ".pred_1"
##
## [[2]]
## [1] ".pred_0" ".pred_1"
##
## [[3]]

```

```
## [1] ".pred_0" ".pred_1"
##
## [[4]]
## [1] ".pred_0" ".pred_1"
##
## [[5]]
## [1] ".pred_0" ".pred_1"
##
## [[6]]
## [1] ".pred_0" ".pred_1"
```

Basically, if you repeat the entire process many times over, the true class for a new, randomly selected observation will be contained within its prediction set $\hat{C}(x) \geq 1 - \alpha = 0.95$ of the time. Where if the true class for the prediction set was “1” and the set was “.pred_1” “.pred_2”, the prediction was correct (95% conf). If the true class was “1” and the set was “.pred_2”, the prediction was incorrect.

b.

```
library(iml)
# Predict function for svm model in predictor environment
predict_function <- function(model, newdata) {
  prob_df <- predict(model, newdata, type = "prob")
  return(prob_df$.pred_1)
}

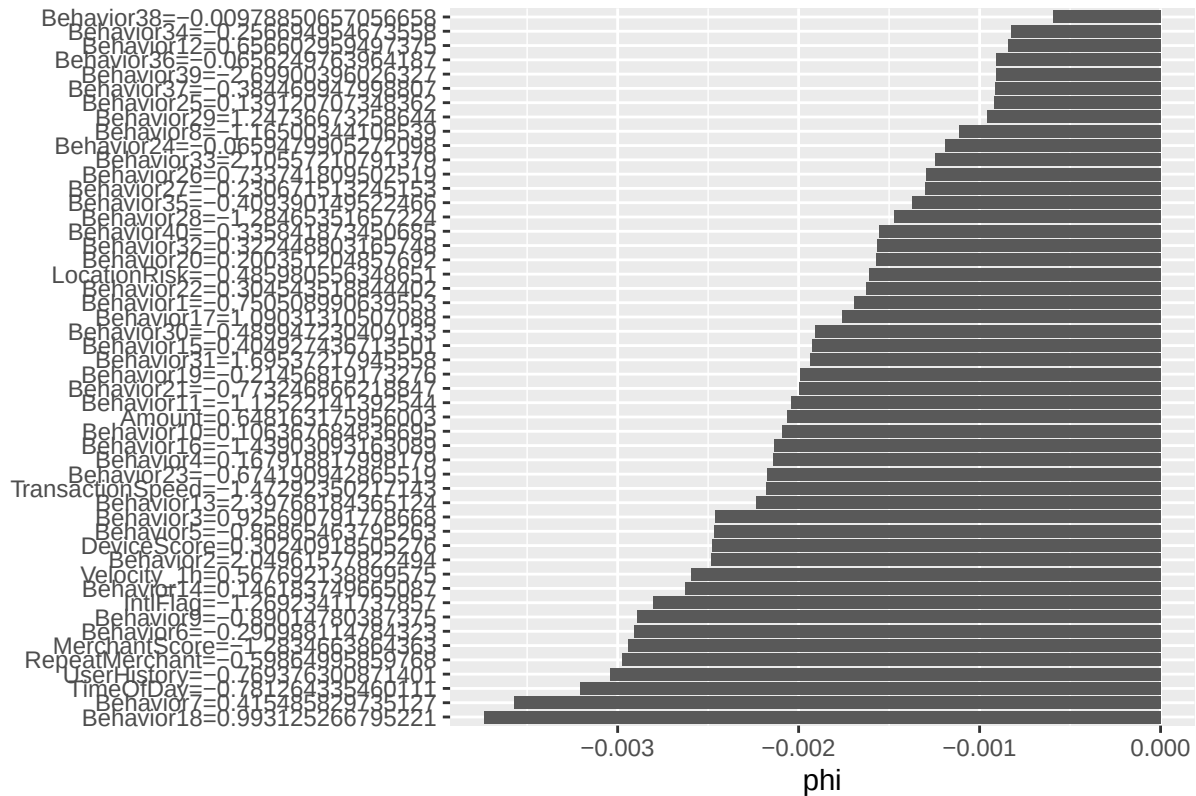
# 2. Create the Predictor environment
# This is the object that handles all subsequent XAI methods (SHAP, LIME, PDP)
predictor <- Predictor$new(
  model = svm_final_fit,
  data = test.[, -51],
  y = test.$class,
  class = "Fraud", # Specify the factor level for the positive class
  predict.function = predict_function
)

importance <- FeatureImp$new(predictor, loss = "ce") # ce = cross-entropy loss
X_sample <- test.[1:20, -51] # Computer could not handle a large sample

# Calculate Shapley values
shap_calc <- Shapley$new(predictor, x.interest = X_sample, sample.size = 100)

# Feature contributions
print(shap_calc$plot() + labs(title = "SHAP Feature Contribution Distribution"))
```

SHAP Feature Contribution Distribution

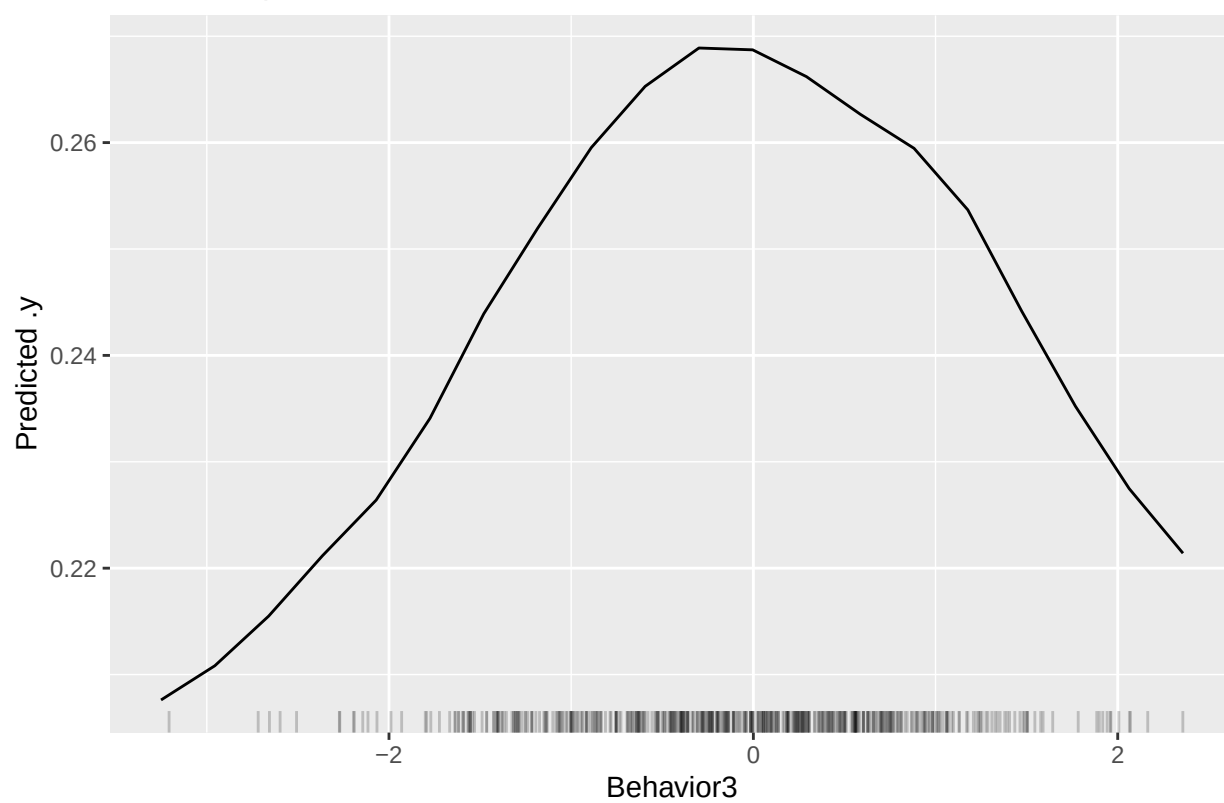


It may be difficult to read because all of the features are tightly packed, but here is the SHAP feature contribution distribution plot. I could not get it to space out more easily, unfortunately.

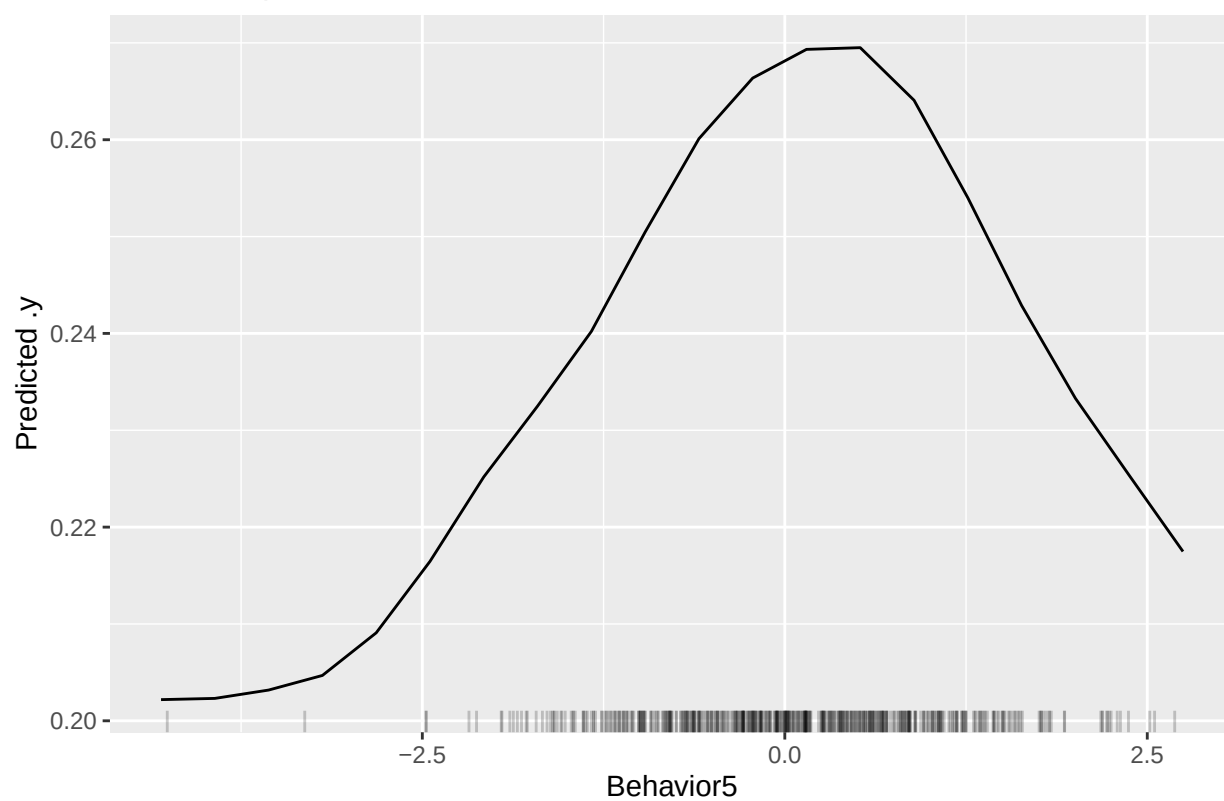
```
for (feat in c("Behavior3","Behavior5","Behavior4")) {
  pdp_plot <- FeatureEffect$new(predictor, feature = feat, method = "pdp")

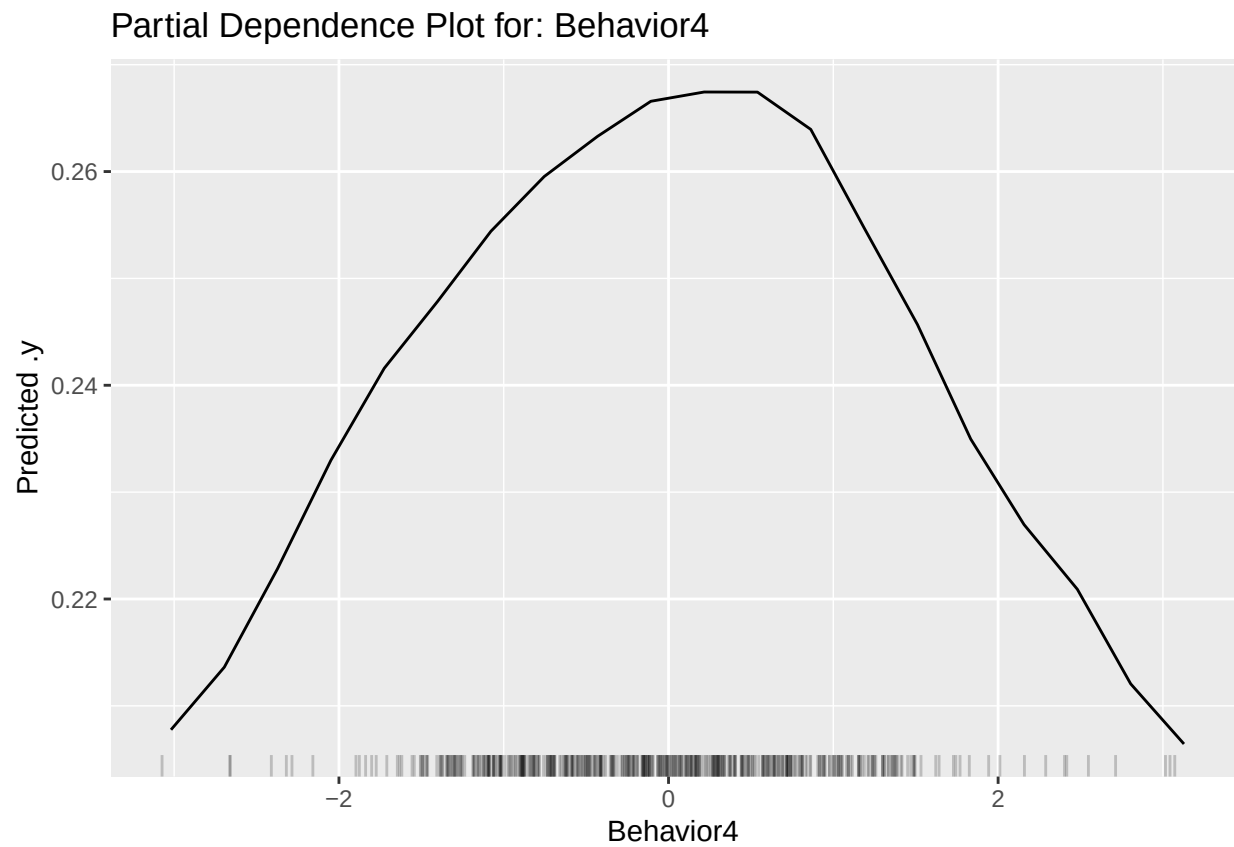
  # Plot the PDP: x-axis is feature value, y-axis is average P(Fraud)
  print(
    pdp_plot$plot() +
    labs(title = paste("Partial Dependence Plot for:", feat))
  )
}
```

Partial Dependence Plot for: Behavior3



Partial Dependence Plot for: Behavior5





Here are partial dependence plots from the top 3 features of the dataset.